

Práctica JavaFX



Programación I

Programa de Ingeniería de Sistemas
y Computación

Armenia, Quindío

2025



UNIVERSIDAD
DEL QUINDÍO
Res. MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN



UNIQUINDÍO
en conexión territorial

www.uniquindio.edu.co

1. JavaFX

JavaFX es una plataforma para desarrollar aplicaciones gráficas de usuario (GUI) en Java. Se utiliza principalmente para crear interfaces de usuario modernas y con estilo en aplicaciones de escritorio, y es el sucesor de las antiguas bibliotecas AWT y Swing.

Características principales:

- 1. Interfaz de usuario moderna:** Permite crear interfaces gráficas con controles como botones, menús, tablas, entre otros, con un diseño visual atractivo.
- 2. Gráficos y animaciones:** Soporta gráficos 2D y 3D, junto con la capacidad de incluir animaciones para crear aplicaciones más dinámicas.
- 3. Escalabilidad:** Las aplicaciones hechas en JavaFX se pueden ejecutar en diferentes dispositivos, como computadoras de escritorio y dispositivos móviles.

1.1 Importancia

JavaFX es crucial para desarrollar interfaces gráficas modernas y atractivas en Java, ofreciendo una plataforma flexible que soporta animaciones, gráficos 2D/3D y multimedia. Permite personalizar interfaces con CSS y separarlas de la lógica usando FXML, lo que facilita la creación de aplicaciones visualmente ricas y bien estructuradas. Además, su integración nativa con Java y soporte multiplataforma asegura un alto rendimiento y portabilidad.

El uso de herramientas como Scene Builder agiliza el diseño visual, mientras que la arquitectura basada en MVC favorece la escalabilidad y mantenimiento del código. JavaFX es la opción recomendada para aplicaciones de escritorio modernas debido a su rendimiento optimizado y continuo desarrollo por parte de Oracle.



1.2 Beneficios

Algunos de los principales beneficios de usar **JavaFX** para el desarrollo de aplicaciones gráficas incluyen:

1. **Interfaz moderna y personalizable:** Permite crear aplicaciones visualmente atractivas con soporte para CSS, lo que facilita el diseño y la personalización de la apariencia.
2. **Separación de lógica y diseño:** FXML ayuda a mantener el código más organizado al separar la estructura de la interfaz de la lógica de negocio, promoviendo una arquitectura limpia y fácil de mantener.
3. **Compatibilidad multiplataforma:** Las aplicaciones JavaFX son ejecutables en diferentes sistemas operativos (Windows, macOS, Linux) sin modificaciones significativas.
4. **Rendimiento gráfico optimizado:** Aprovecha la aceleración por hardware para gráficos 2D/3D y animaciones, lo que garantiza un rendimiento fluido en aplicaciones intensivas en gráficos.
5. **Facilidad de desarrollo visual:** Herramientas como **Scene Builder** permiten diseñar interfaces de manera visual, acelerando el desarrollo y reduciendo la cantidad de código manual.
6. **Multimedia integrada:** Soporte nativo para reproducir audio y video, lo que facilita la creación de aplicaciones multimedia sin dependencias adicionales.



2. Herramientas y extensiones

2.1 Herramientas

Las herramientas necesarias para trabajar el proyecto son:

JavaFX es crucial para desarrollar interfaces gráficas modernas y atractivas en Java, ofreciendo una plataforma flexible que soporta animaciones, gráficos 2D/3D y multimedia. Permite personalizar interfaces con CSS y separarlas de la lógica usando FXML, lo que facilita la creación de aplicaciones visualmente ricas y bien estructuradas. Además, su integración nativa con Java y soporte multiplataforma asegura un alto rendimiento y portabilidad.

El uso de herramientas como **Scene Builder** agiliza el diseño visual, mientras que la arquitectura basada en MVC favorece la escalabilidad y mantenimiento del código. JavaFX es la opción recomendada para aplicaciones de escritorio modernas debido a su rendimiento optimizado y continuo desarrollo por parte de Oracle.

Herramienta	Descripción	Link descarga
 IntelliJ	IntelliJ IDEA es el JetBrains IDE para el desarrollo profesional en Java.	Link
 Scene Builder	JavaFX Scene Builder es una herramienta visual para diseñar interfaces gráficas (GUI) en aplicaciones JavaFX, permitiendo arrastrar y soltar componentes sin escribir código XML (eXtensible Markup Language).	Link

2.3 Código de la práctica del proyecto empresa en el repositorio

Pueden descargar el código del proyecto accediendo al enlace que se muestra a continuación ([link repositorio](#)).

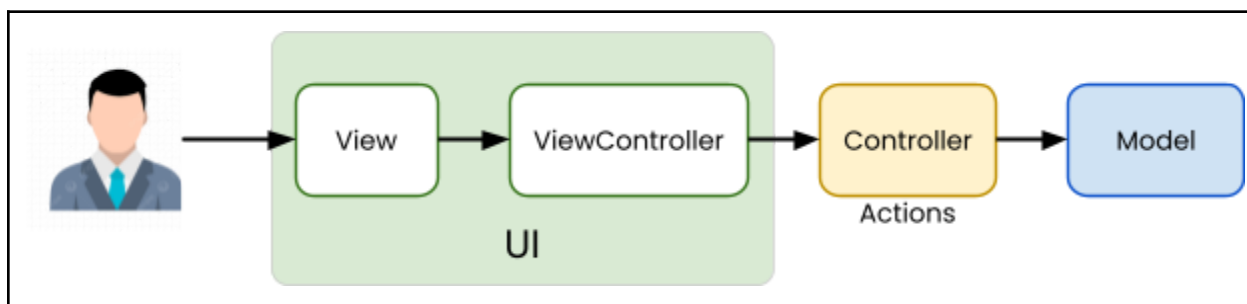
Repositorio	Link proyecto
 RaulYulbrayner Owns this repository Committed to this repository in the past day	

3. Arquitectura

La imagen muestra una arquitectura de software basada en el patrón **MVC (Modelo-Vista-Controlador)**, utilizada para el diseño de interfaces gráficas. Representa el flujo de interacción entre el usuario (ubicado en el extremo izquierdo) y las distintas capas del sistema, separando claramente las responsabilidades entre la interfaz gráfica, el control de la lógica y los datos.

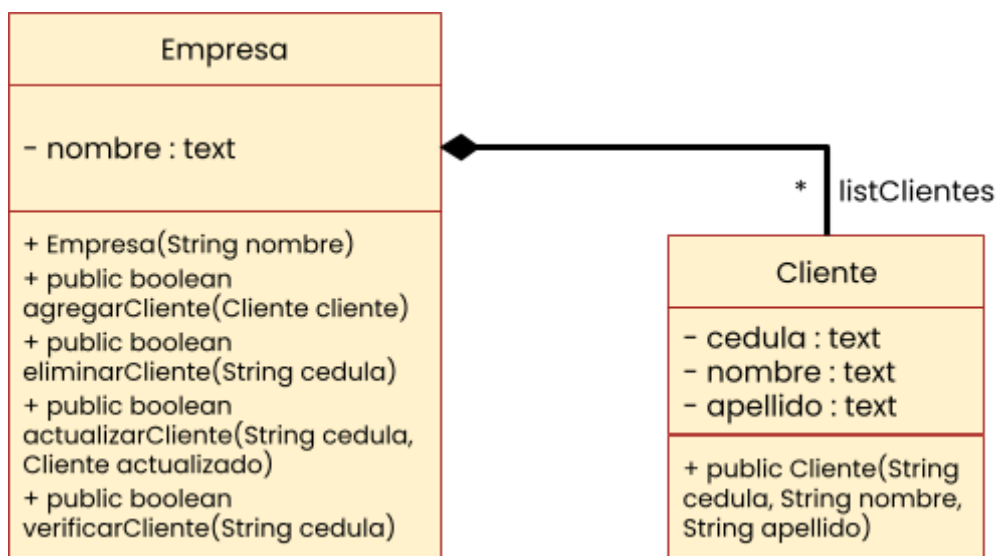
Las capas de Vista (View) y ViewController están agrupadas bajo un bloque común llamado UI (Interfaz de Usuario). La Vista (View) se encarga de mostrar la interfaz visual al usuario, definida mediante FXML, lo que sugiere que se utilizan descripciones estructuradas para los elementos gráficos de la aplicación. El ViewController gestiona la interacción entre la Vista y el Controlador (Controller), recibiendo las entradas del usuario y decidiendo qué acciones tomar, mientras mantiene la lógica de la interfaz separada del resto del sistema.

El Controlador (Controller) maneja las acciones del usuario, recibe los eventos desde la Vista a través del ViewController y coordina con el Modelo (Model) para ejecutar acciones o actualizar datos. El Modelo (Model) contiene la lógica de negocio o los datos de la aplicación, y es responsable de procesar la información requerida.

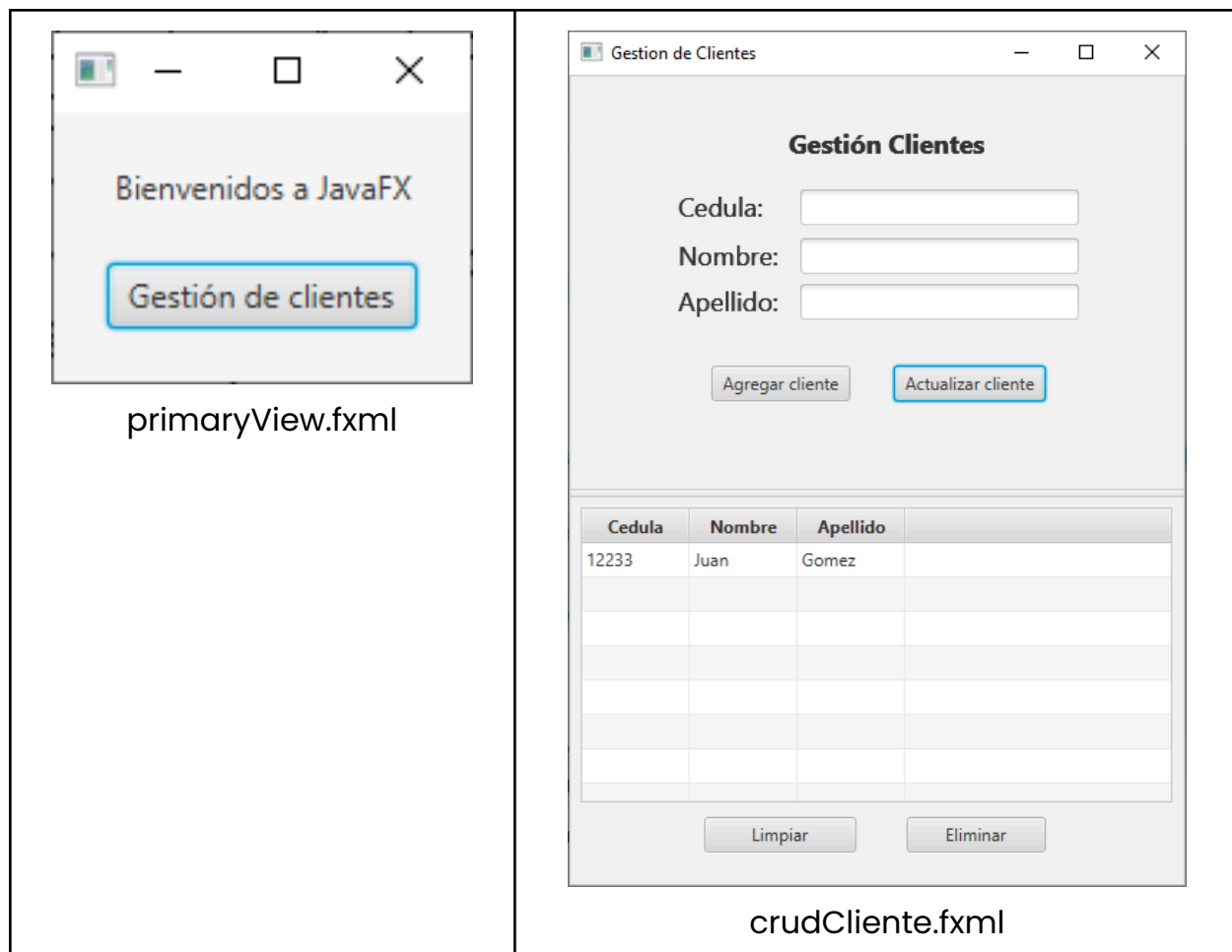


4. Practica

La práctica a partir de este diagrama de clases consiste en implementar una gestión de clientes dentro de una empresa, utilizando los conceptos de programación orientada a objetos (POO). La clase Empresa contendrá una lista de clientes, permitiendo realizar diversas operaciones sobre ellos, como agregar, eliminar, actualizar y verificar clientes.



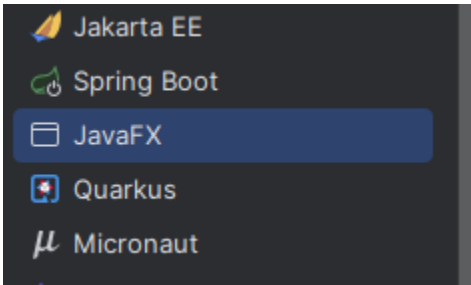
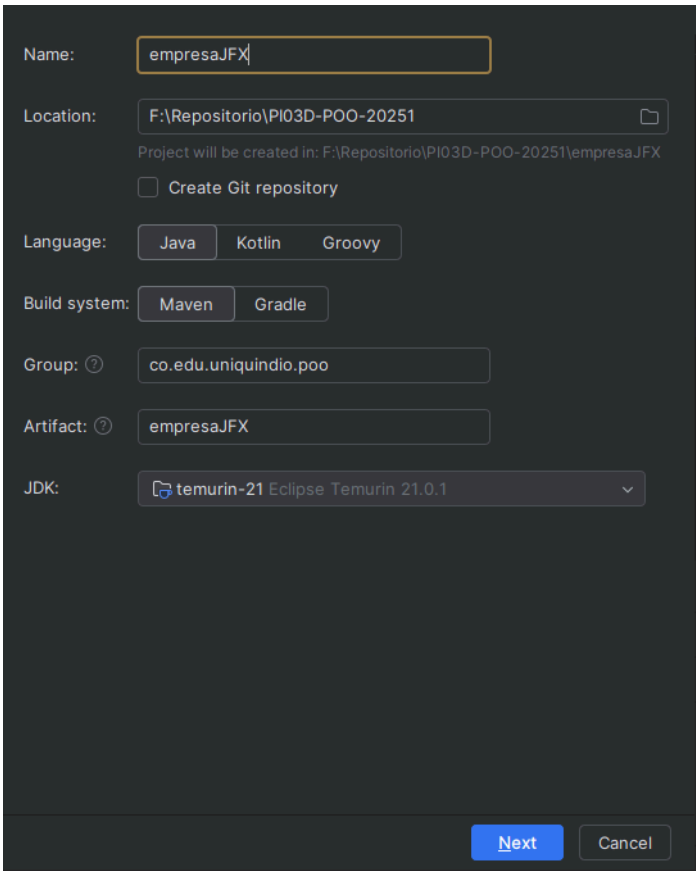
Como resultado de esta práctica, el estudiante desarrollará y adquirirá conocimientos en JavaFX para crear interfaces de usuario en sus proyectos. A través de esta experiencia, implementará una aplicación de gestión de clientes que permitirá administrar toda la información de los clientes de una empresa. Esto incluirá funcionalidades como agregar, actualizar, eliminar y verificar clientes en una lista, lo que proporcionará una solución práctica y funcional para la organización de datos en una empresa real.

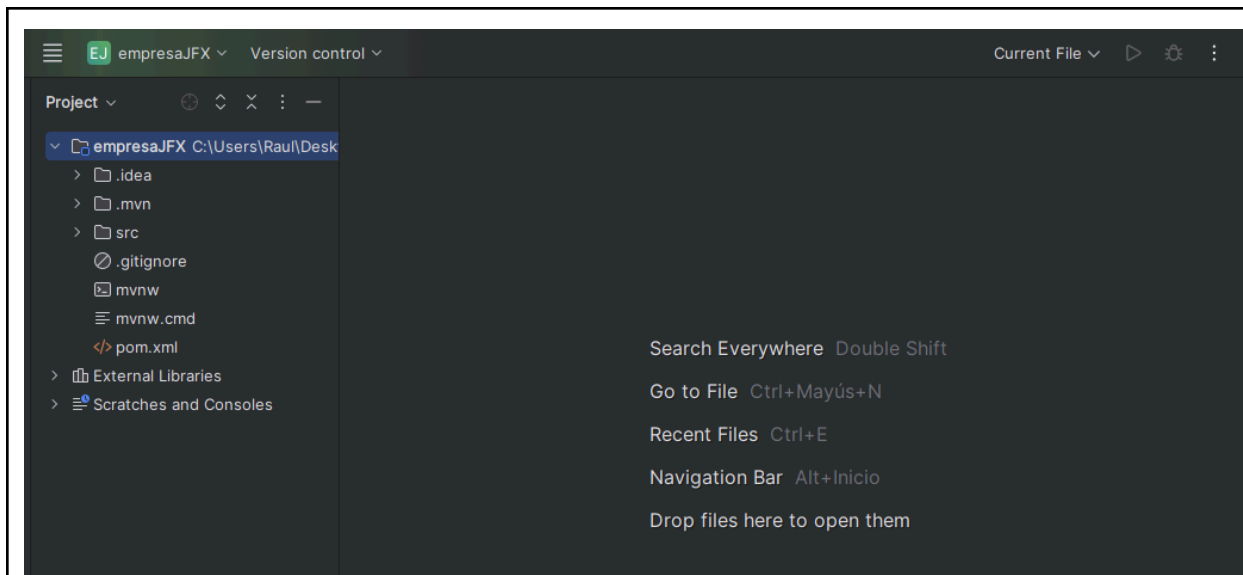


La imagen muestra dos interfaces gráficas diseñadas en JavaFX utilizando archivos FXML.

- **primaryView.fxml:** Es la ventana principal de bienvenida de la aplicación, con un botón titulado "Gestión de clientes". Al presionar este botón, el usuario será redirigido a la vista de gestión de clientes.
- **crudCliente.fxml:** Es la interfaz principal para la gestión de clientes. En esta ventana, el usuario puede ingresar los datos de un cliente, como cédula, nombre y apellido, mediante campos de texto. Además, cuenta con botones para agregar y actualizar clientes. En la parte inferior se encuentra una tabla donde se muestra la lista de clientes registrados, con sus respectivas cédulas, nombres y apellidos. También hay botones adicionales para limpiar los campos de entrada o eliminar un cliente de la lista seleccionada.

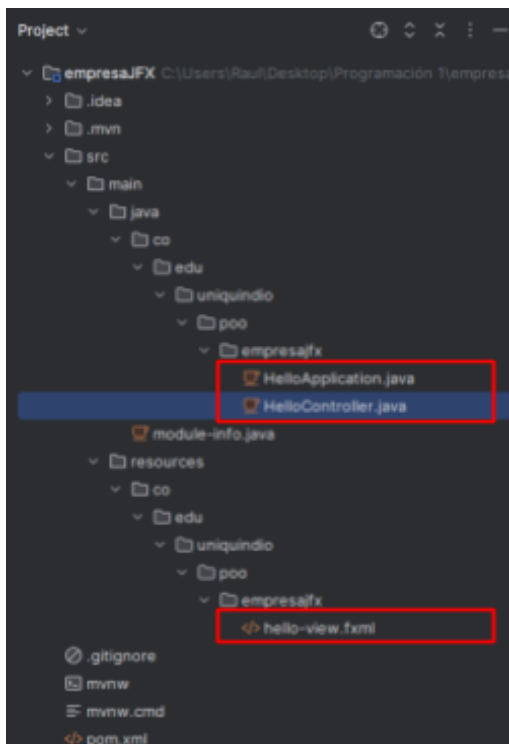
4.1 Creación del proyecto javaFX en IntelliJ

1. File -> New -> Project	
1.1 Crear el proyecto en javaFX.	
1.2 Configurar: • Name: empresaJFX • Location: “seleccionan el lugar donde van a guardar el proyecto” • Language: Java • Builder system: Maven • Group: co.edu.uniquindio.poo • Artifact: empresaJFX • JDK: Seleccionar el jdk, ej: 21 Una vez terminado las configuraciones le dan NEXT	
1.3 Additional Libraries: en este punto, no deben realizar nada, solo deben dar clic en Create para crear el proyecto.	
2. Visualización del proyecto javaFX creado	



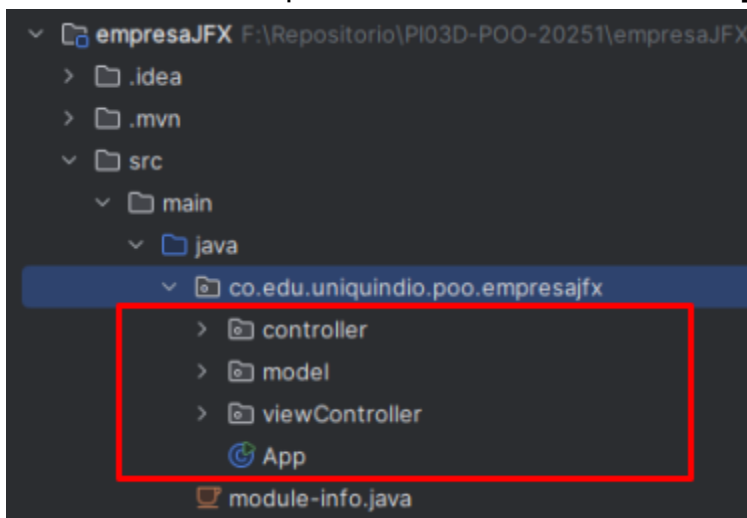
3. Eliminación de archivos

Eliminar los archivos dar clic en: **HelloApplication.java**, **HelloController.java** Y **hello-view.fxml**.



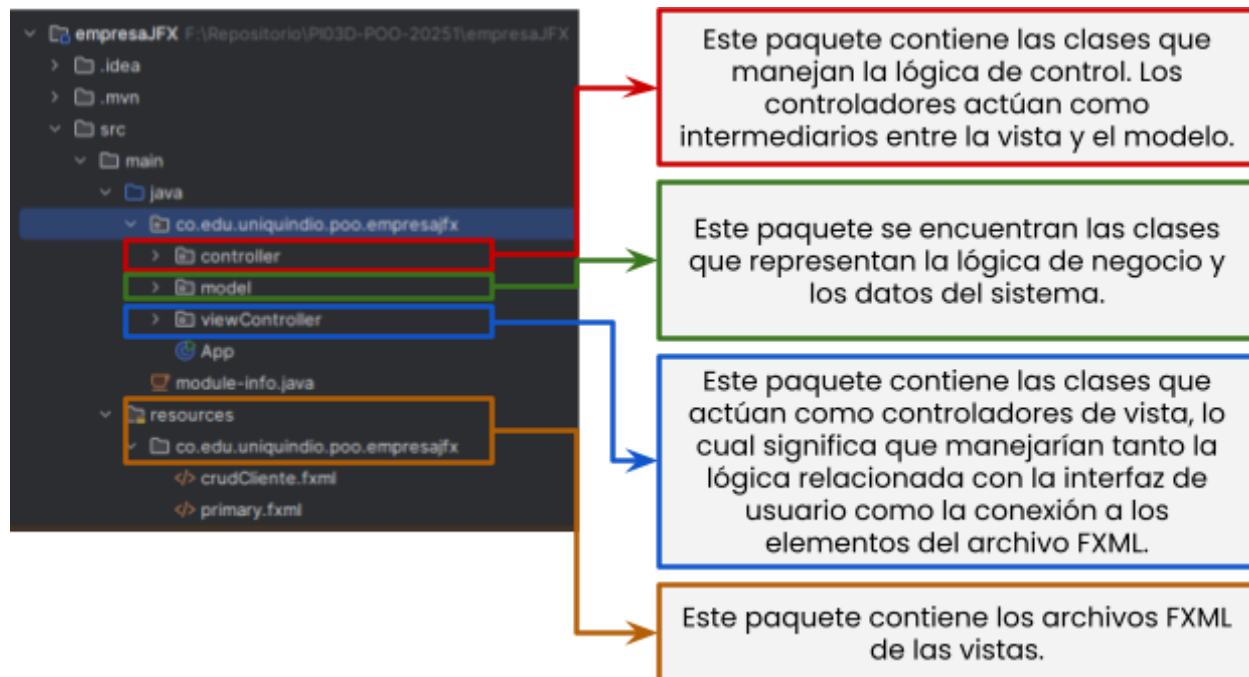
4. Creación de las carpetas - controller, model y viewController

Para crear las carpetas damos clic en **New -> package**



4.2 Estructura de paquetes

La estructura de paquetes en un proyecto Java es fundamental porque organiza el código de manera clara y modular, facilitando el mantenimiento, la escalabilidad y la colaboración en equipo. Seguir una estructura bien definida, como la separación entre controladores, modelos y vistas (MVC), es esencial. A continuación, se presenta la estructura de paquetes utilizada en la práctica:

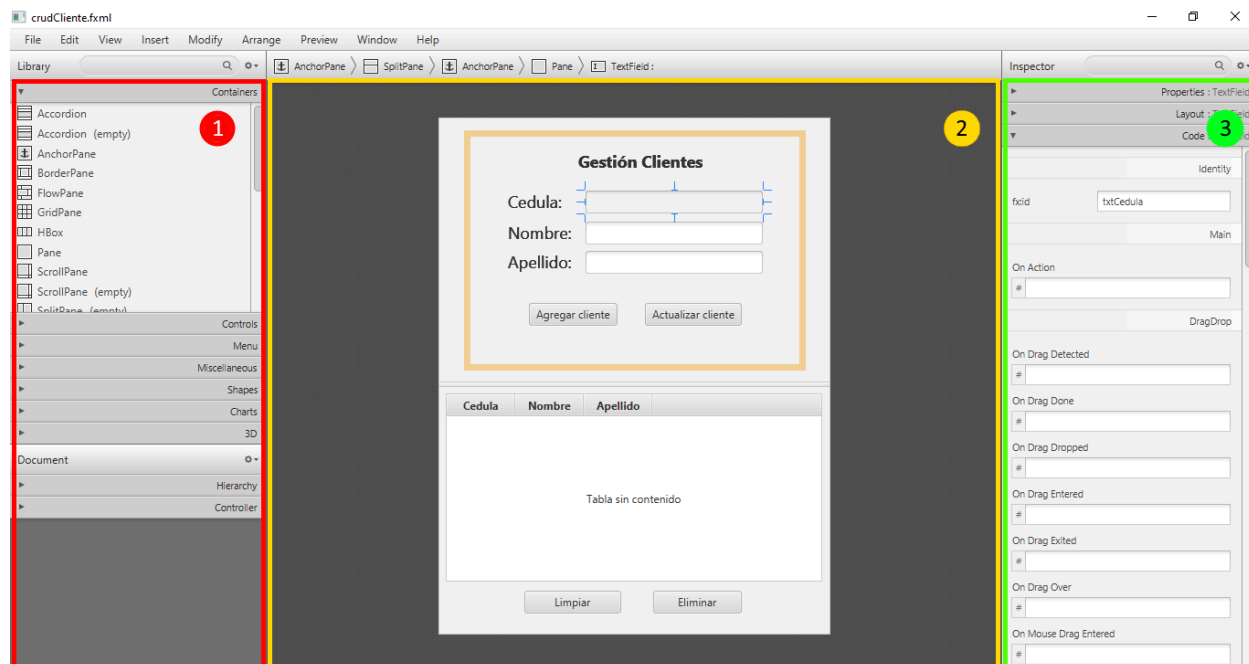


Para concluir:

- **Controller:** Lógica de control y respuesta a eventos de la vista.
- **Model:** Lógica de negocio y gestión de datos.
- **ViewController:** Controladores específicos para manejar la lógica de la vista en interacción directa con el FXML.
- **Resources:** Archivos de recursos como FXML, imágenes o CSS que complementan la interfaz de usuario.

4.3 Creación de la UI en Scene Builder

La herramienta Scene Builder nos permite crear de manera sencilla la interfaz gráfica que necesitamos. En esta práctica, hemos diseñado la interfaz de "Gestión de Clientes", como se muestra en la figura. La herramienta se divide en tres zonas principales: (1) la zona de componentes, (2) el área de trabajo o canvas, y (3) la zona de propiedades.



5. Código

En esta sección de la práctica se presenta el código, con el objetivo de que los estudiantes lo sigan paso a paso. El código está dividido en **seis subítems**, cada uno de los cuales incluye el código correspondiente a la vista (interfaz gráfica), el **viewController**, el **controller**, el **modelo** y la clase **App**.

5.1 Código clase App

Modificar la clase “**App.java**”

1. App.java

```
package co.edu.uniquindio.poo.empresajfx;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.scene.layout.AnchorPane;
import javafx.stage.Stage;
import java.io.IOException;
import co.edu.uniquindio.poo.empresajfx.model.Cliente;
import co.edu.uniquindio.poo.empresajfx.model.Empresa;
import co.edu.uniquindio.poo.empresajfx.viewController.ClienteViewController;
import co.edu.uniquindio.poo.empresajfx.viewController.PrimaryController;

/**
 * JavaFX App
 */
public class App extends Application {

    private Stage primaryStage;
    public static Empresa empresa = new Empresa("UQ");

    @Override
    public void start(Stage primaryStage) throws IOException {
        this.primaryStage = primaryStage;
        this.primaryStage.setTitle("Gestion de Clientes");
        openViewPrincipal();
    }

    private void openViewPrincipal() {
        inicializarData();
    }
}
```

```

        try {
            FXMLLoader loader = new FXMLLoader();
            loader.setLocation(App.class.getResource("primary.fxml"));
            javafx.scene.layout.VBox rootLayout = (javafx.scene.layout.VBox)
loader.load();
            PrimaryController primaryController = loader.getController();
            primaryController.setApp(this);

            Scene scene = new Scene(rootLayout);
            primaryStage.setScene(scene);
            primaryStage.show();
        } catch (IOException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        launch();
    }

    public void openCrudCliente() {
        try {
            FXMLLoader loader = new FXMLLoader();
            loader.setLocation(App.class.getResource("crudCliente.fxml"));
            AnchorPane rootLayout = (AnchorPane) loader.load();
            ClienteViewController clienteViewController =
loader.getController();
            clienteViewController.setApp(this);

            Scene scene = new Scene(rootLayout);
            primaryStage.setScene(scene);
            primaryStage.show();
        } catch (IOException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }

    //servicios
    public void inicializarData(){
        Cliente cliente = new Cliente("12233", "juan", "apellido");
        empresa.agregarCliente(cliente);
    }
}

```


5.2 Código paquete model

Estas clases se debe crear en el paquete **“model”**

1. Empresa.java

```
package co.edu.uniquindio.poo.empresajfx.model;

import java.util.Collection;
import java.util.LinkedList;

public class Empresa {
    private String nombre;
    private Collection<Cliente> clientes;

    public Empresa(String nombre) {
        this.nombre = nombre;
        clientes = new LinkedList<>();
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public Collection<Cliente> getClientes() {
        return clientes;
    }

    public void setClientes(Collection<Cliente> clientes) {
        this.clientes = clientes;
    }

    public boolean agregarCliente(Cliente cliente) {
        boolean centinela = false;
        if (!verificarCliente(cliente.getCedula())) {
            clientes.add(cliente);
            centinela = true;
        }
        return centinela;
    }
}
```



```

public boolean eliminarCliente(String cedula) {
    boolean centinela = false;
    for (Cliente cliente : clientes) {
        if (cliente.getCedula().equals(cedula)) {
            clientes.remove(cliente);
            centinela = true;
            break;
        }
    }
    return centinela;
}

public boolean actualizarCliente(String cedula, Cliente actualizado) {
    boolean centinela = false;
    for (Cliente cliente : clientes) {
        if (cliente.getCedula().equals(cedula)) {
            cliente.setCedula(actualizado.getCedula());
            cliente.setNombre(actualizado.getNombre());
            cliente.setApellido(actualizado.getApellido());
            centinela = true;
            break;
        }
    }
    return centinela;
}

public boolean verificarCliente(String cedula) {
    boolean centinela = false;
    for (Cliente cliente : clientes) {
        if (cliente.getCedula().equals(cedula)) {
            centinela = true;
        }
    }
    return centinela;
}
}

```

2. Cliente.java

```

package co.edu.uniquindio.poo.empresajfx.model;

public class Cliente {
    private String cedula, nombre, apellido;

    public Cliente(String cedula, String nombre, String apellido) {

```

```

        this.cedula = cedula;
        this.nombre = nombre;
        this.apellido = apellido;
    }

    public String getCedula() {
        return cedula;
    }

    public void setCedula(String cedula) {
        this.cedula = cedula;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
    }
}

```

5.3 Código de la interfaz

Estas clases se debe crear en el paquete **“resources”**

1. primary.fxml

```

<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.*?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<VBox alignment="CENTER" spacing="20.0" xmlns="http://javafx.com/javafx/8"
xmlns:fx="http://javafx.com/fxml/1"

```

```
fx:controller="co.edu.uniquindio.poo.empresajfx.viewController.PrimaryContro
ller">
    <children>
        <Label text="Bienvenidos a JavaFX" />
        <Button fx:id="primaryButton" onAction="#onOpenCrudCliente"
text="Gestión de clientes" />
    </children>
    <padding>
        <Insets bottom="20.0" left="20.0" right="20.0" top="20.0" />
    </padding>
</VBox>
```

2. crudCliente.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.effect.*?>
<?import javafx.scene.text.*?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<AnchorPane maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity"
minWidth="-Infinity" prefHeight="566.0" prefWidth="429.0"
xmlns="http://javafx.com/javafx/8" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="co.edu.uniquindio.poo.empresajfx.viewController.ClienteViewCo
ntroller">
    <children>
        <SplitPane dividerPositions="0.5159574468085106"
orientation="VERTICAL" prefHeight="566.0" prefWidth="429.0">
            <items>
                <AnchorPane minHeight="0.0" minWidth="0.0"
prefHeight="288.0" prefWidth="401.0">
                    <children>
                        <Pane layoutX="34.0" layoutY="20.0"
prefHeight="249.0" prefWidth="360.0" style="-fx-border-image-width: 1px;">
                            <effect>
                                <Blend />
                            </effect>
                            <children>
                                <Label layoutX="118.0" layoutY="14.0"
text="Gestión Clientes">
                                    <font>
                                        <Font name="System Bold" size="18.0"
/>
                                </font>
                            </children>
                        </Pane>
                    </children>
                </AnchorPane>
            </items>
        </SplitPane>
    </children>
</AnchorPane>
```

```

        </Label>
        <Label layoutX="41.0" layoutY="58.0"
prefHeight="17.0" prefWidth="64.0" text="Cedula:">
            <font>
                <Font size="18.0" />
            </font>
        </Label>
        <Label layoutX="41.0" layoutY="92.0"
text="Nombre:">
            <font>
                <Font size="18.0" />
            </font>
        </Label>
        <Label layoutX="41.0" layoutY="124.0"
text="Apellido:">
            <font>
                <Font size="18.0" />
            </font>
        </Label>
        <TextField fx:id="txtCedula" layoutX="126.0"
layoutY="59.0" prefHeight="25.0" prefWidth="194.0" />
        <TextField fx:id="txtNombre" layoutX="126.0"
layoutY="93.0" prefHeight="25.0" prefWidth="194.0" />
        <TextField fx:id="txtApellido"
layoutX="126.0" layoutY="125.0" prefHeight="25.0" prefWidth="194.0" />
        <Button fx:id="btbAgregarCliente"
layoutX="64.0" layoutY="182.0" mnemonicParsing="false"
onAction="#onAgregarCliente" text="Agregar cliente" />
        <Button fx:id="btnActualizarCliente"
layoutX="191.0" layoutY="182.0" mnemonicParsing="false"
onAction="#onActualizarCliente" text="Actualizar cliente" />
    </children>
</Pane>
</children>
</AnchorPane>
<AnchorPane minHeight="0.0" minWidth="0.0"
prefHeight="271.0" prefWidth="645.0">
    <children>
        <TableView fx:id="tblListCliente" layoutX="7.0"
layoutY="7.0" prefHeight="206.0" prefWidth="412.0">
            <columns>
                <TableColumn fx:id="tbcCedula"
prefWidth="75.0" text="Cedula" />
                <TableColumn fx:id="tbcNombre"
prefWidth="75.0" text="Nombre" />
            </columns>
        </TableView>
    </children>

```

```

                                <TableColumn fx:id="tbcApellido"
prefWidth="75.0" text="Apellido" />
                                </columns>
                                </TableView>
                                <Button fx:id="btnLimpiar" layoutX="93.0"
layoutY="223.0" mnemonicParsing="false" onAction="#onLimpiar"
prefHeight="25.0" prefWidth="106.0" text="Limpiar" />
                                <Button fx:id="btnEliminar" layoutX="234.0"
layoutY="223.0" mnemonicParsing="false" onAction="#onEliminar"
prefHeight="25.0" prefWidth="97.0" text="Eliminar" />
                                </children>
                                </AnchorPane>
                                </items>
                                </SplitPane>
                                </children>
                                </AnchorPane>

```

5.4 Código viewController

Estas clases se debe crear en el paquete **"viewController"**

1. PrimaryController.java

```

package co.edu.uniquindio.poo.empresajfx.viewController;

import java.net.URL;
import java.util.ResourceBundle;
import co.edu.uniquindio.poo.empresajfx.App;
import javafx.fxml.FXML;
import javafx.scene.control.Button;

public class PrimaryController {
    @FXML
    private ResourceBundle resources;

    App app;
    @FXML
    private URL location;

    @FXML
    private Button primaryButton;

    @FXML
    void onOpenCrudCliente() {

```



```

        app.openCrudCliente();
    }

    public void setApp(App app) {
        this.app = app;
    }

    @FXML
    void initialize() {

    }
}

```

2. ClienteViewController.java

```

package co.edu.uniquindio.poo.empresajfx.viewController;

import java.net.URL;
import java.util.ResourceBundle;

import co.edu.uniquindio.poo.empresajfx.App;
import co.edu.uniquindio.poo.empresajfx.controller.ClienteController;
import co.edu.uniquindio.poo.empresajfx.model.Cliente;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.Button;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.TextField;

public class ClienteViewController {

    ClienteController clienteController;
    ObservableList<Cliente> listClientes =
FXCollections.observableArrayList();
    Cliente selectedCliente;

    @FXML
    private ResourceBundle resources;

    @FXML
    private URL location;

```



```
@FXML
private TextField txtNombre;

@FXML
private Button btnLimpiar;

@FXML
private TableView<Cliente> tblListCliente;

@FXML
private Button btnEliminar;

@FXML
private Button btnActualizarCliente;

@FXML
private TableColumn<Cliente, String> tbcNombre;

@FXML
private TextField txtApellido;

@FXML
private TableColumn<Cliente, String> tbcApellido;

@FXML
private Button btnAgregarCliente;

@FXML
private TableColumn<Cliente, String> tbcCedula;

@FXML
private TextField txtCedula;
private App app;

@FXML
void onAgregarCliente() {
    agregarCliente();
}

@FXML
void onActualizarCliente() {
    actualizarCliente();
}

@FXML
```

```

void onLimpiar() {
    limpiarSeleccion();
}

@FXML
void onEliminar() {
    eliminarCliente();
}

@FXML
void initialize() {
    this.app=app;
    clienteController = new ClienteController(app.empresa);
    initView();
}

private void initView() {
    // Traer los datos del cliente a la tabla
    initDataBinding();

    // Obtiene la lista
    obtenerClientes();

    // Limpiar la tabla
    tblListCliente.getItems().clear();

    // Agregar los elementos a la tabla
    tblListCliente.setItems(listClientes);

    // Seleccionar elemento de la tabla
    listenerSelection();
}

private void initDataBinding() {
    tbcCedula.setCellValueFactory(cellData → new
SimpleStringProperty(cellData.getValue().getCedula()));
    tbcNombre.setCellValueFactory(cellData → new
SimpleStringProperty(cellData.getValue().getNombre()));
    tbcApellido.setCellValueFactory(cellData → new
SimpleStringProperty(cellData.getValue().getApellido()));
    // Usamos SimpleObjectProperty para manejar Double y Integer
correctamente
}

private void obtenerClientes() {

```

```

        listClientes.addAll(clienteController.obtenerListaClientes());
    }

    private void listenerSelection() {
tblListCliente.getSelectionModel().selectedItemProperty().addListener((obs,
oldSelection, newSelection) → {
        selectedCliente = newSelection;
        mostrarInformacionCliente(selectedCliente);
    });
}

    private void mostrarInformacionCliente(Cliente cliente) {
        if (cliente ≠ null) {
            txtCedula.setText(cliente.getCedula());
            txtNombre.setText(cliente.getNombre());
            txtApellido.setText(cliente.getApellido());
        }
    }

    private void agregarCliente() {
        Cliente cliente = buildCliente();
        if (clienteController.crearCliente(cliente)) {
            listClientes.add(cliente);
            limpiarCamposCliente();
        }
    }

    private Cliente buildCliente() {
        Cliente cliente = new Cliente(txtCedula.getText(),
txtNombre.getText(), txtApellido.getText());
        return cliente;
    }

    private void eliminarCliente() {
        if (clienteController.eliminarCliente(txtCedula.getText())) {
            listClientes.remove(selectedCliente);
            limpiarCamposCliente();
            limpiarSeleccion();
        }
    }

    private void actualizarCliente() {

        if (selectedCliente ≠ null &&

```

```

clienteController.actualizarCliente(selectedCliente.getCedula(),
buildCliente())) {

    int index = listClientes.indexOf(selectedCliente);
    if (index ≥ 0) {
        listClientes.set(index, buildCliente());
    }

    tblListCliente.refresh();
    limpiarSeleccion();
    limpiarCamposCliente();
}

private void limpiarSeleccion() {
    tblListCliente.getSelectionModel().clearSelection();
    limpiarCamposCliente();
}

private void limpiarCamposCliente() {
    txtCedula.clear();
    txtNombre.clear();
    txtApellido.clear();
}

public void setApp(App app) {
    this.app = app;
}
}

```

5.5 Código controller

Estas clases se debe crear en el paquete **“controller”**

1. ClienteController.java

```

package co.edu.uniquindio.poo.empresajfx.controller;

import co.edu.uniquindio.poo.empresajfx.model.Cliente;
import co.edu.uniquindio.poo.empresajfx.model.Empresa;
import java.util.Collection;

public class ClienteController {

```

```

Empresa empresa;

public ClienteController(Empresa empresa) {
    this.empresa = empresa;
}

public boolean crearCliente(Cliente cliente) {
    return empresa.agregarCliente(cliente);
}

public Collection<Cliente> obtenerListaClientes() {
    return empresa.getClientes();
}

public boolean eliminarCliente(String cedula) {
    return empresa.eliminarCliente(cedula);
}

public boolean actualizarCliente(String cedula, Cliente cliente) {
    return empresa.actualizarCliente(cedula, cliente);
}
}

```

5.6 Código module-info

Modificar la clase **“module-info.java”**

1. module-info.java

```

module co.edu.uniquindio.poo.empresajfx {
    requires javafx.controls;
    requires javafx.fxml;

    opens co.edu.uniquindio.poo.empresajfx to javafx.fxml;
    exports co.edu.uniquindio.poo.empresajfx;
    exports co.edu.uniquindio.poo.empresajfx.viewController;
    opens co.edu.uniquindio.poo.empresajfx.viewController to javafx.fxml;
}

```



UNIQUEINDÍO, en conexión territorial

Carrera 15 Calle 12 Norte Tel: (606) 7 35 93 00 Armenia - Quindío - Colombia

www.uniquindio.edu.co